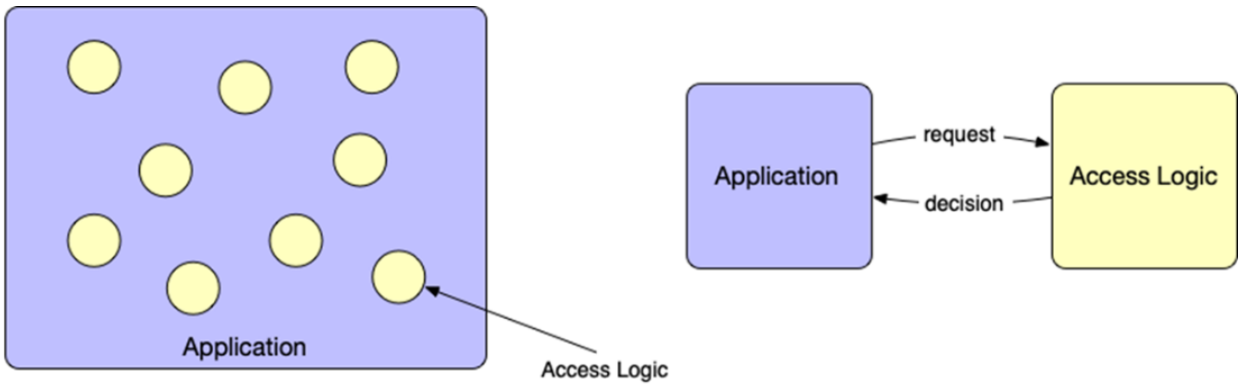


Figure 1.1 Embedding access logic throughout application code (left) creates tight coupling. Externalizing authorization into a separate component (right) makes access policies explicit, decouples decision-making from application behavior, and enables scalable, auditable access control.

This book will focus on policy-based access control (PBAC), which decouples access logic from sapplication code as shown in figure 1.1. Externalizing the logic allows for flexible, fine-grained, and auditable access decisions that are easier to author, maintain, and enforce consistently at scale. PBAC uses a policy engine to evaluate policies and determine whether a specific access request should be allowed or denied. PBAC has been around for some time, but has only recently started to be more widely used as new, easier-to-use policy languages and better, general-purpose policy engines have been developed.

1.2 Challenges with traditional approaches to authorization

Robust authorization practices are crucial to how the modern internet works. But authorization has been used in computing for decades. For example, Unix file permissions allow files and directories to be owned by an account and shared with groups of users or everyone on the system. Later, access control lists (ACLs) expanded on file permissions to allow more flexible control of actions, while role-based access control (RBAC) made groups more flexible and easier to manage.

All these access control methods have one thing in common: they are static. They use static stores of explicit permissions, groups, and roles to control